

Machine Learning Model

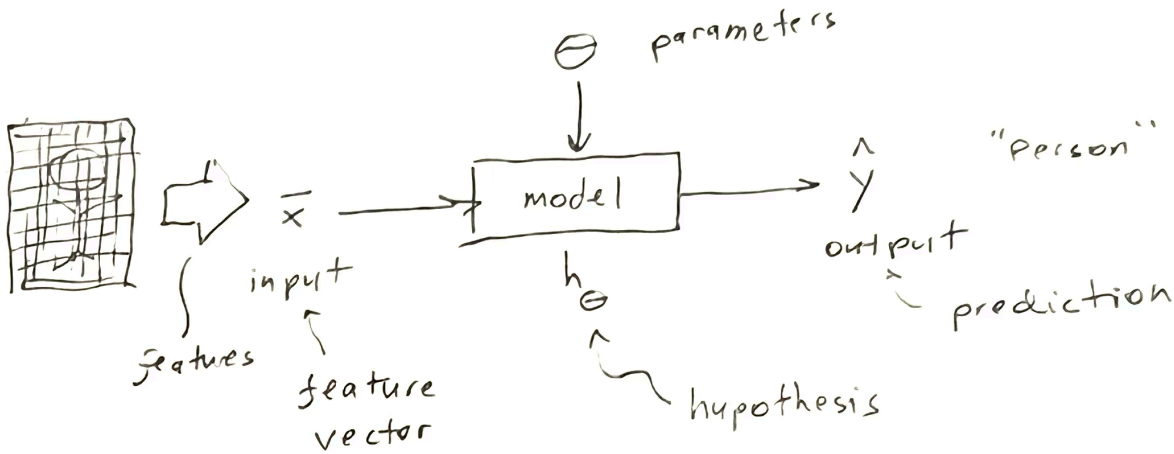
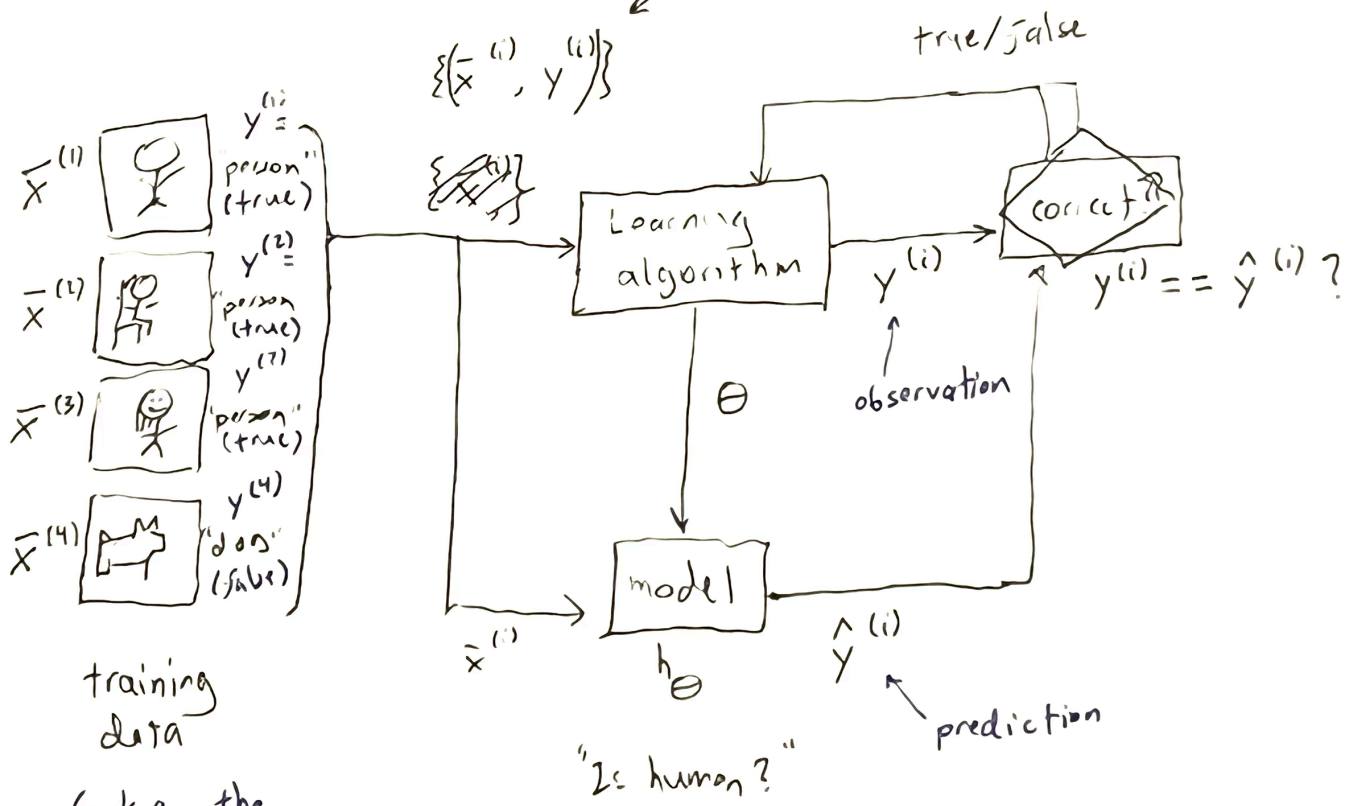


Figure 1: An ML model

Figure 1 shows a model in the "prediction phase" meaning it has already been trained. Training sets the parameters Θ .



(a.k.a., the "ground truth")

Figure 2: human classifier training

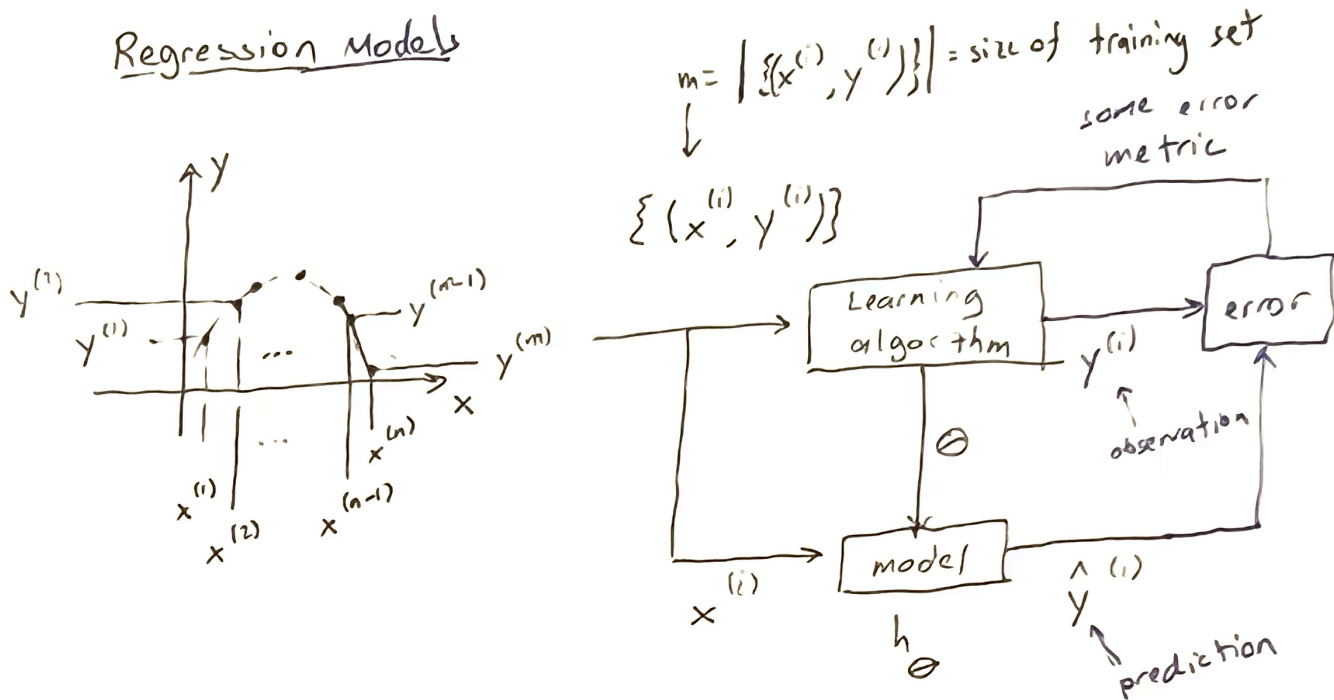
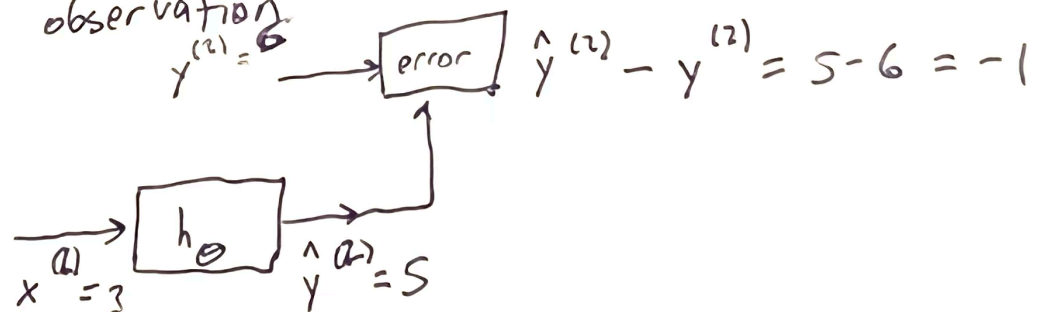
Regression Models

Figure 3: training a regression model

How do we measure error?

Similar to measuring dispersion. Variance & MAD characterize deviations from the mean.

Error is deviation between the prediction and the observation

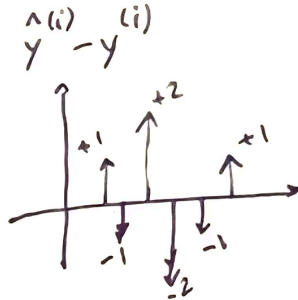


$$e^{(i)} = \text{error} = \hat{y}^{(i)} - y^{(i)}$$

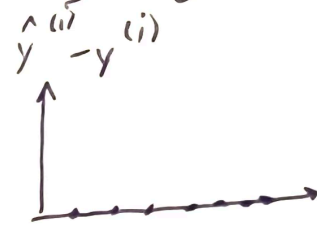
Figure 4: calculating error

Measuring Error

As with measuring dispersion, summing deviations results in positive errors canceling negative errors.



(a)



(b)

Figure 5: Two datasets with cancelling errors.

$$\text{sum error} = \sum_{i=1}^m \epsilon_i = (\hat{y}_1 - y_1) + (\hat{y}_2 - y_2) + \dots + (\hat{y}^{(n)} - y^{(n)})$$

For (a) $\sum_{i=1}^m \epsilon_i$ is given by

$$\sum_{i=1}^m \epsilon_i = +1 + (-1) + 2 + (-2) + (-1) + 1 = 0$$

For (b) $\sum_{i=1}^m \epsilon_i$ is

$$\sum_{i=1}^m \epsilon_i = 0 + 0 + \dots + 0 = 0$$

Both (a) & (b) have 0 sum error, but intuitively a model that outputs ^{predicted} values that oscillate around the observed values exhibits more error than one always give the correct predictions.

Measuring Error (cont.)

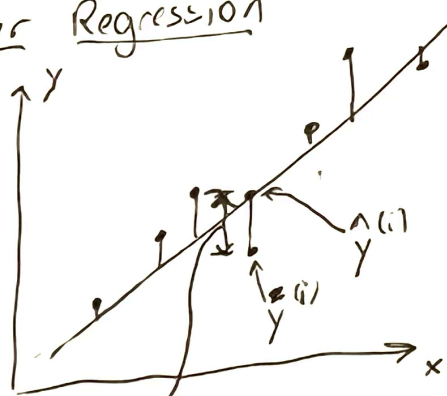
The problem of devising an error metric has similar motivations as used when considering dispersion metrics. We can achieve differentiability and tends to converge with more samples by using the mean sum squared error:

$$\text{Mean Square Error} = \frac{1}{m} \sum_{i=1}^m (y_i^{(i)} - \hat{y}^{(i)})^2 \quad (2)$$

Mean square ^{error} has the same problem ^{as} variance in that its units are square of units for $y^{(i)}$ and $\hat{y}^{(i)}$. We can solve this problem in the same way as a standard deviation.

$$\text{Root mean square error (RMSE)} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i^{(i)} - \hat{y}^{(i)})^2} \quad (3)$$

Linear Regression



linear approximation to the observations $\{y^{(i)}\}$.

$$E^{(i)} = \hat{y}^{(i)} - y^{(i)}$$

With linear regression, we want to find the error that minimizes mean square error.

Aside: minimizing sum square error simultaneously

minimizes mean square error and Root Mean Square Error.

(4)

Linear Regression

In the context of machine learning, the function we minimize by adjust parameters is called a cost function $J(\theta)$

cost function model parameters

For linear regression the model is given by

$$y^{(i)} = h_{\theta}(\bar{x}^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots + \theta_n x_n^{(i)} \quad (5)$$

This model take an n -dimensional point and outputs a scalar.

In linear algebra we express ~~it~~ thus

$$\bar{X} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, \quad \hat{y} = \theta^T \bar{x}$$

mean square error

The divide by 2 makes the derivative cleaner. (see next page)

cost function

$$J(\bar{\theta}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\bar{x}^{(i)}) - y^{(i)})^2$$

In matrix notation

$$J(\bar{\theta}) = \frac{1}{2m} (X\bar{\theta} - \bar{y})^T (X\bar{\theta} - \bar{y}) \quad (7)$$

X = design matrix =

$$\begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix}$$

2nd feature vector in the training data set

m feature vectors in the training data set

$$\bar{y} = \begin{bmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \vdots \\ \hat{y}^{(m)} \end{bmatrix}$$

m observed values